

Fibonacci Search Method for Univariate Optimization

Sulagna Roy

2026-09-12

Contents

Overview	1
The Objective Function	1
The Fibonacci Search Function	2
Explanation, step by step	3
Running the Search	5
Interpreting the result	5
Visualizing the Convergence	6
Reading the plot	6
Summary	7

Overview

This document walks through an implementation of the **Fibonacci search method**, a classical technique for finding the minimizer of a *unimodal* function over a closed interval $[a, b]$ without using derivatives.

The method works by successively shrinking a bracketing interval that is guaranteed to contain the minimum. At each step, it evaluates the function at two interior points chosen according to ratios of consecutive **Fibonacci numbers**, and discards the sub-interval that cannot contain the minimum based on which of the two function values is larger. Because the two interior points at each iteration are chosen so that one of them can be reused in the next iteration, the method only requires **one new function evaluation per iteration** (after the first two), which makes it very efficient when function evaluations are expensive.

The number of iterations is controlled entirely by how many Fibonacci numbers are needed to shrink the interval to within a target tolerance, `tol`.

The Objective Function

```
# Objective function
f <- function(x) {
  ((x - 2) * (x - 8))^2
}
```

The function being minimized is

$$f(x) = [(x - 2)(x - 8)]^2.$$

This is a quartic (degree-4) polynomial. It touches zero at $x = 2$ and $x = 8$ (its two roots, each a double root because of the square), and is strictly positive everywhere else. As a result, f has **two local minima**, at $x = 2$ and $x = 8$, separated by a local maximum somewhere in between. Over the search interval used below, $[1, 5]$, only the minimum at $x = 2$ lies inside the bracket, so that is the one the algorithm will converge to.

The Fibonacci Search Function

```
# Fibonacci search function
fib_search <- function(f, a, b, tol = 1e-4) {

  # Generate Fibonacci numbers
  fib <- c(1, 1)
  while (fib[length(fib)] < (b - a) / tol) {
    fib <- c(fib, fib[length(fib)] + fib[length(fib) - 1])
  }

  history <- data.frame(
    a = a,
    b = b
  )

  N <- length(fib)

  # Initial points
  x1 <- a + fib[N - 2] / fib[N] * (b - a)
  x2 <- a + fib[N - 1] / fib[N] * (b - a)

  f1 <- f(x1)
  f2 <- f(x2)

  for (k in 1:(N - 2)) {

    if (f1 > f2) {

      a <- x1
      x1 <- x2
      f1 <- f2

      x2 <- a + fib[N - k - 1] / fib[N - k] * (b - a)
      f2 <- f(x2)
    }
  }
}
```

```

    } else {

      b <- x2
      x2 <- x1
      f2 <- f1

      x1 <- a + fib[N - k - 2] / fib[N - k] * (b - a)
      f1 <- f(x1)
    }

    history <- rbind(
      history,
      data.frame(a = a, b = b)
    )
  }

  list(
    xmin = (a + b) / 2,
    fmin = f((a + b) / 2),
    interval = c(a, b),
    history = history
  )
}

```

Explanation, step by step

1. Generating the Fibonacci sequence (lines under “Generate Fibonacci numbers”)

```

fib <- c(1, 1)
while (fib[length(fib)] < (b - a) / tol) {
  fib <- c(fib, fib[length(fib)] + fib[length(fib) - 1])
}

```

The classic Fibonacci sequence $1, 1, 2, 3, 5, 8, 13, \dots$ is built up until the last term exceeds $(b - a)/\text{tol}$. This ratio determines *how many* Fibonacci numbers (and therefore how many iterations) are needed to shrink the original interval width $(b - a)$ down to a final width smaller than tol . A smaller tol requires more Fibonacci numbers and hence more iterations.

2. Tracking the interval history

```

history <- data.frame(a = a, b = b)

```

An empty-ish data frame is initialized with the starting interval endpoints. This will be used later to record how a and b evolve at every iteration, which is what makes it possible to visualize the convergence.

3. Choosing the initial two interior points

```

N <- length(fib)
x1 <- a + fib[N - 2] / fib[N] * (b - a)
x2 <- a + fib[N - 1] / fib[N] * (b - a)

```

Instead of bisectioning the interval, Fibonacci search places two points x_1 and x_2 inside $[a, b]$ using ratios of Fibonacci numbers:

$$x_1 = a + \frac{F_{N-2}}{F_N}(b - a), \quad x_2 = a + \frac{F_{N-1}}{F_N}(b - a).$$

Since $F_{N-2} < F_{N-1}$, we always have $x_1 < x_2$. The function is evaluated at both points (`f1 <- f(x1)`, `f2 <- f(x2)`).

4. The main iteration loop

```
for (k in 1:(N - 2)) {
  if (f1 > f2) {
    a <- x1; x1 <- x2; f1 <- f2
    x2 <- a + fib[N - k - 1] / fib[N - k] * (b - a)
    f2 <- f(x2)
  } else {
    b <- x2; x2 <- x1; f2 <- f1
    x1 <- a + fib[N - k - 2] / fib[N - k] * (b - a)
    f1 <- f(x1)
  }
  history <- rbind(history, data.frame(a = a, b = b))
}
```

This is the core comparison-and-shrink step, repeated $N - 2$ times:

- **If $f(x_1) > f(x_2)$:** the minimum cannot lie to the left of x_1 (the function is higher there than at x_2), so the left endpoint is moved up to x_1 (`a <- x1`). The old x_2 becomes the new x_1 (reusing its already-computed function value, `f1 <- f2`, so no new evaluation is needed there), and a *new* x_2 is computed using the next ratio of Fibonacci numbers, requiring exactly one new function evaluation (`f2 <- f(x2)`).
- **Otherwise ($f(x_1) \leq f(x_2)$):** symmetric logic — the minimum cannot lie to the right of x_2 , so the right endpoint moves down to x_2 (`b <- x2`), the old x_1 becomes the new x_2 (reusing `f1` as `f2`), and a new x_1 is computed and evaluated.

After each iteration, the updated bracket $[a, b]$ is appended to `history`. Because the interval shrinks by a Fibonacci ratio at every step rather than being simply halved, the interval width shrinks *faster* than bisection while still needing only one function evaluation per iteration.

5. Returning the result

```
list(
  xmin = (a + b) / 2,
  fmin = f((a + b) / 2),
  interval = c(a, b),
  history = history
)
```

After the loop finishes, `a` and `b` have converged to (nearly) the same point. The function returns:

- `xmin`: the midpoint of the final tiny interval, used as the best estimate of the minimizer.
- `fmin`: the objective function evaluated at that midpoint.
- `interval`: the final $[a, b]$ bracket.
- `history`: the full record of how the bracket shrank at every iteration, useful for diagnostics and plotting.

Running the Search

```
# Run Fibonacci search
result <- fib_search(f, a = 1, b = 5, tol = 1e-4)

print(result)
```

```
## $xmin
## [1] 1.999957
##
## $fmin
## [1] 6.697797e-08
##
## $interval
## [1] 1.999914 2.000000
##
## $history
##      a      b
## 1  1.000000 5.000000
## 2  1.000000 3.472136
## 3  1.000000 2.527864
## 4  1.583592 2.527864
## 5  1.583592 2.167184
## 6  1.806504 2.167184
## 7  1.944272 2.167184
## 8  1.944272 2.082039
## 9  1.944272 2.029417
## 10 1.976794 2.029417
## 11 1.976794 2.009317
## 12 1.989217 2.009317
## 13 1.996894 2.009317
## 14 1.996894 2.004572
## 15 1.996894 2.001639
## 16 1.998706 2.001639
## 17 1.998706 2.000518
## 18 1.999396 2.000518
## 19 1.999827 2.000518
## 20 1.999827 2.000259
## 21 1.999827 2.000086
## 22 1.999914 2.000086
## 23 1.999914 2.000000
```

Interpreting the result

Running the search on $[a, b] = [1, 5]$ with $\text{tol} = 1\text{e-}4$, the printed output above shows:

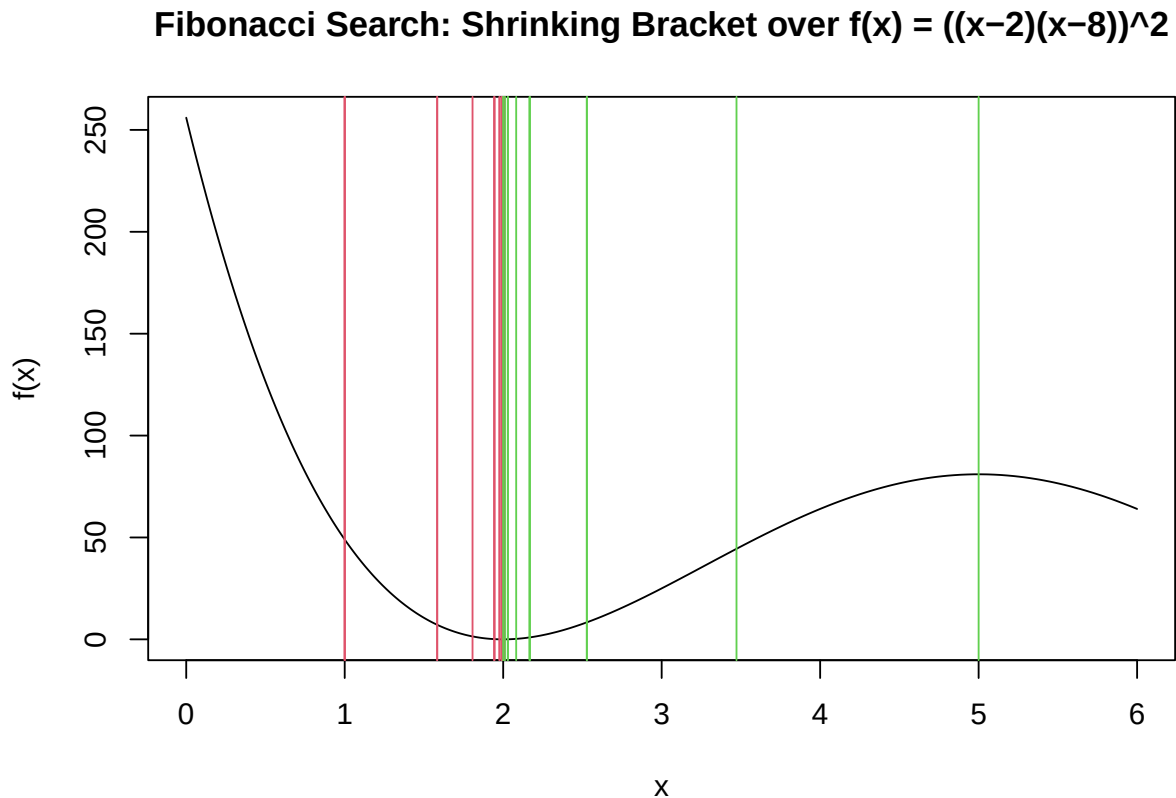
- $\text{xmin} \approx 1.999957$ — extremely close to the true minimizer at $x = 2$, as expected since $f(2) = 0$ exactly.
- $\text{fmin} \approx 6.698 \times 10^{-8}$ — a value very near zero, confirming we are essentially at the minimum.
- `interval` — the final bracket, of width on the order of 10^{-4} , matching the requested tolerance.
- `history` — a 23-row data frame, meaning the search ran for that many iterations (including the initial interval) before the bracket shrank below `tol`. Notice how a steadily increases and b steadily decreases

from iteration to iteration, and how the *gap* $b - a$ shrinks by roughly the ratio of consecutive Fibonacci numbers each time rather than by a constant factor of one-half (as bisection would).

Visualizing the Convergence

```
h <- result$history

u <- seq(0, 6, length = 500)
plot(u, f(u), type = 'l',
     main = "Fibonacci Search: Shrinking Bracket over f(x) = ((x-2)(x-8))^2",
     xlab = "x", ylab = "f(x)")
for (i in 1:nrow(h)) {
  abline(v = h[i, 1], col = 2) # left endpoint a, in red
  abline(v = h[i, 2], col = 3) # right endpoint b, in green
}
```



Reading the plot

The black curve is $f(x) = ((x-2)(x-8))^2$ plotted over $x \in [0, 6]$. Because of the squared product of two linear terms, the curve touches zero at $x = 2$ (visible as the dip near the left side of the plot) and would touch zero again at $x = 8$ if the plotting range were extended further right; within $[0, 6]$ we only see the rising side of that second dip near the right edge of the plot.

The vertical lines trace the algorithm’s progress:

- **Red lines** mark the left endpoint a at each iteration.
- **Green lines** mark the right endpoint b at each iteration.

At iteration 1, the red and green lines are far apart (spanning the original $[1, 5]$ interval). With each subsequent iteration, a red line appears slightly to the right of the previous one and a green line appears slightly to the left, so the lines visibly “funnel in” toward $x = 2$. By the final iterations, the red and green lines are essentially on top of one another, visually confirming that the bracket has collapsed onto the minimizer located at the bottom of the left dip in the curve.

Summary

This script implements a textbook Fibonacci search:

1. It precomputes enough Fibonacci numbers to guarantee the interval shrinks below the desired tolerance.
2. It places two test points per iteration using Fibonacci ratios, reusing one function evaluation from the previous step so that each iteration costs only one new evaluation of $\mathbf{f}$.
3. It repeatedly discards the half of the interval that cannot contain the minimum, based on comparing the two test-point function values.
4. It records the interval history for diagnostic plotting, and returns the midpoint of the final bracket as the estimated minimizer.

For the quartic test function $f(x) = ((x - 2)(x - 8))^2$ on $[1, 5]$, the method correctly converges to $x \approx 2$, the local (and, within this bracket, global) minimizer, with an objective value essentially at zero.